

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-210БВ-25

Студент: Сивец Д.Ю.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 04.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через `pipe1`, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через `pipe2`. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число число число<newline>». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `float`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)` — создаёт дочерний процесс как копию родительского; в родителе возвращает `pid` ребёнка, в ребёнке `0`.
- `int pipe(int pipefd[2])` — создаёт неименованный канал: `pipefd[0]` для чтения, `pipefd[1]` для записи; вызывается до `fork`, чтобы ребёнок унаследовал концы.
- `int dup2(int oldfd, int newfd)` — переназначает дескриптор: в ветке ребёнка `dup2(p[0], 0)` делает стандартный ввод читающим концом канала.
- `int execl(const char *path, const char *arg, ..., (char *)NULL)` — заменяет образ процесса программой `child`, передавая имя файла через `argv`.
- `pid_t waitpid(pid_t pid, int *status, 0)` — ожидает завершения ребёнка; макросы `WIFEXITED/WEXITSTATUS` извлекают код выхода.
- `int open(const char *pathname, int flags, mode_t mode)` — создаёт и открывает файл результата (`O_WRONLY|O_CREAT|O_TRUNC`, права `0644`).
- `ssize_t read(int fd, void *buf, size_t count)` и `ssize_t write(int fd, const void *buf, size_t count)` — весь ввод-вывод программы, включая передачу команд через канал.
- `int close(int fd)` — закрытие дескрипторов; закрытие всех пишущих концов канала служит сигналом EOF читающему процессу.
- `void _exit(int status) / exit(int status)` — завершение процесса с кодом, которым ребёнок сообщает родителю об аварии.

Родитель

1. читает первую строку (имя файла) посимвольно, чтобы не проглотить байты сверх перевода строки;
2. создаёт канал;
3. fork;
4. в ветке ребёнка закрывает пишущий конец, dup2 читает конца в stdin, закрывает оригинал, exec1 программы child с именем файла в argv;
5. в ветке родителя закрывает читающий конец и циклом read(0)/write(p[1]) перекачивает команды пользователя в канал;
6. по достижении EOF на консоли (Ctrl+D) закрывает пишущий конец, после чего read ребёнка возвращает 0.

Ребёнок

1. читает stdin побайтово, собирает токены до пробельных символов, преобразует atof и накапливает сумму в типе float;
2. по окончании ввода форматирует сумму и записывает в файл, созданный через open.

Родитель дожидается ребёнка через waitpid и при ненулевом коде выхода завершается тем же кодом.

Оговорки: pipe2 не создаётся, так как необходимость обратного канала в варианте отсутствует, ошибки передаются кодом выхода; валидация имени файла делегирована вызову open; точность вывода фиксирована — 6 знаков после точки, округление арифметическое; накопление ведётся в float согласно условию, погрешность не превышает одного ulp.

Код программы

parent.c

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

static void put_err(const char* s) {
    int len = 0;
    while (s[len]) len++;
    write(2, s, len);
}

static int read_line(int fd, char* buf, int cap) {
    int len = 0;
    char c;
    while (len < cap - 1 && read(fd, &c, 1) == 1) {
        if (c == '\n') break;
        buf[len++] = c;
    }
    buf[len] = '\0';
    return len;
}

int main() {
    char fname[256];
    if (read_line(0, fname, sizeof(fname)) <= 0) {
```

```

        put_err("parent: expected filename on first line\n");
        return 1;
    }
    int p[2];
    if (pipe(p) == -1) {
        put_err("parent: pipe failed\n");
        return 1;
    }
    pid_t pid = fork();
    if (pid == -1) {
        put_err("parent: fork failed\n");
        return 1;
    }
    if (pid == 0) {
        close(p[1]);
        if (dup2(p[0], 0) == -1) {
            put_err("child branch: dup2 failed\n");
            _exit(1);
        }
        close(p[0]);
        execl("./child", "child", fname, (char *)NULL);
        put_err("parent: exec ./child failed\n");
        _exit(1);
    }
    close(p[0]);
    char buf[4096];
    ssize_t n;
    while ((n = read(0, buf, sizeof(buf))) > 0) {
        if (write(p[1], buf, (size_t)n) != n) {
            put_err("parent: write to pipe failed\n");
            break;
        }
    }
    close(p[1]);
    int status;
    if (waitpid(pid, &status, 0) == -1) {
        put_err("parent: waitpid failed\n");
        return 1;
    }
    if (WIFEXITED(status)) {
        int code = WEXITSTATUS(status);
        if (code != 0) {
            put_err("parent: child reported error\n");
            return code;
        }
        return 0;
    }
    put_err("parent: child terminated abnormally\n");
    return 1;
}

```

child.c

```

#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

static void put_err(const char* s) {
    int len = 0;
    while (s[len]) len++;
    write(2, s, len);
}

static void write_sum(int fd, float sum, int decimals) {
    if (decimals > 7) {
        decimals = 7;
    }
    char buf[64]; int len = 0;
    if (sum < 0) { buf[len++] = '-'; sum = -sum; }
    long whole = (long)sum;
    long scale = 1;
    for (int i = 0; i < decimals; i++) scale *= 10;
    long frac = (long)((float)(sum - (float)whole) * (float)scale + 0.5);
    if (frac == scale) { whole += 1; frac = 0; }
    char tmp[48]; int t = 0;
    if (whole == 0) tmp[t++] = '0';
    while (whole > 0) { tmp[t++] = (char)('0' + whole % 10); whole /= 10; }
    for (int i = t - 1; i >= 0; i--) buf[len++] = tmp[i];
    if (decimals > 0) {
        buf[len++] = '.';
        t = 0;
        for (int i = 0; i < decimals; i++) { tmp[t++] = (char)('0' + frac % 10); frac
/= 10; }
        for (int i = t - 1; i >= 0; i--) buf[len++] = tmp[i];
    }
    buf[len++] = '\n';
    write(fd, buf, len);
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        put_err("usage: child filename\n");
        return 1;
    }
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        put_err("child: cannot open file\n");
        return 1;
    }
    float sum = 0.0f;
    char tok[64];
    int tlen = 0;
    char c;
    while (read(0, &c, 1) == 1) {
        if (c == ' ' || c == '\n' || c == '\t' || c == '\r') {
            if (tlen > 0) { /* токен закончился */
                tok[tlen] = '\0';
                sum += (float)atof(tok);
            }
            tlen = 0;
        }
    }
}

```

```

        tlen = 0;
    }
    } else if (tlen < 63) {
        tok[tlen++] = c;
    }
}
if (tlen > 0) {
    tok[tlen] = '\0';
    sum += (float)atof(tok);
}
write_sum(fd, sum, 6);
close(fd);
return 0;
}

```

Протокол работы программы

Тестирование:

Тест 1 (основной сценарий):

```

$ ./parent
out.txt
1 2 3
1.2 1.4 1.6 1.8
<Ctrl+D>
$ cat out.txt
17.999998
$ echo $?
0

```

Тест 2 (провал exec, child переименован):

```

$ mv child child.bak
$ ./parent
out.txt
1 2
parent: exec ./child failed
parent: write to pipe failed
parent: child reported error
$ echo $?
1

```

Тест 3 (неудающийся open у ребёнка):

```

$ ./parent
/no/dir/x.txt
child: cannot open file
1 2
parent: write to pipe failed
parent: child reported error
$ echo $?
1

```

Тест 4 (пустая первая строка):

```
$ ./parent
<Ctrl+D>
parent: expected filename on first line
$ echo $?
1
```

Тест 5 (имя файла с пробелами – допустимое имя):

```
$ ./parent
my file.txt
1 2 3
<Ctrl+D>
$ cat "my file.txt"
6.000000
```

Вывод

В ходе работы приобретены практические навыки управления процессами и обмена данными через неименованный канал в Linux: реализована схема варианта 2, в которой родитель передаёт команды ребёнку через `pipe1`, подключённый к стандартному вводу, а ребёнок записывает сумму в созданный им файл; отработаны обработка ошибок всех системных вызовов и согласованное завершение процессов через код выхода и `waitpid`. При выполнении возникли и были решены следующие проблемы: настройка окружения сборки под WSL, взаимная блокировка из-за незакрытых концов канала (ошибка `EPIPE` у родителя при преждевременно завершившемся ребёнке), а также округление `float` при выводе (17.999998 вместо 18 как следствие шага сетки типа).